

CHANDIGARH UNIVERSITY

Department of Computer Science & Engineering

AI / ML Mini Project Report

DocuTrust

A Self-Correcting Retrieval-Augmented Generation (RAG) Platform

Corrective RAG (CRAG) · LangGraph · Cross-Encoder Reranking · FastAPI · ChromaDB · MongoDB

Project Title	DocuTrust — Self-Correcting Enterprise RAG Platform (Corrective RAG / CRAG)
Student Name	Devansh Singh Raghuvanshi
College Name	Chandigarh University
Branch & Year	B.E. Computer Science & Engineering — Final Year
Email	dsraghuvanshi2002@gmail.com
Phone Number	+91 9140405360
Submission Date	01 July 2026

File: Devansh_DocuTrust.pdf

2. Project Overview

2.1 Problem Statement

Standard Retrieval-Augmented Generation (RAG) systems retrieve the top-k document chunks by vector similarity and pass them directly to a language model, with no check on whether the retrieved content is actually relevant. When retrieval fails — due to ambiguous phrasing, a question the document set does not cover, or a weak embedding match — the model still produces a fluent, confident-sounding answer from irrelevant context. Retrieval failure and retrieval success are therefore indistinguishable from the outside, which is a serious reliability gap for enterprise document Q&A, where an unverified wrong answer is worse than no answer at all.

2.2 Objective

To design and implement a self-correcting RAG pipeline based on the Corrective RAG (CRAG) pattern: every retrieval is scored by a cross-encoder reranker and graded against tuned relevance thresholds before any answer is generated. Retrievals graded as weak trigger an automatic query-rewrite-and-retry loop (capped to guarantee termination), and retrievals that remain poor escalate to a disclosed web-search fallback. The goal is a system that measurably reduces confident, ungrounded answers compared with a naive RAG baseline, while remaining fully traceable — every citation in an answer maps back to a specific source chunk and page.

2.3 Technologies Used

Technology / Library	Role	Purpose
Python 3.12	Language	Core programming language for the entire backend
LangGraph 1.2	Orchestration	Explicit state machine for the CRAG retrieve→grade→correct loop
FastAPI 0.138	Backend	REST API + WebSocket server for live trace streaming
ChromaDB 1.5	Vector store	Persistent storage and similarity search over document chunks
sentence-transformers	Embeddings	BAAI/bge-small-en-v1.5 bi-encoder for chunk/query embedding
BAAI/bge-reranker-v2-m3	Reranking	Cross-encoder that grades retrieval relevance (the CRAG core)
MongoDB	Database	Stores run traces, citations, and document metadata
Anthropic Claude / Ollama	LLM	Pluggable generation backend (cloud API or local model)
Tavily API	Web search	Fallback search when internal documents are insufficient
Docker / Docker Compose	Deployment	One-command containerised setup
pytest	Testing	25-test automated suite covering ingestion, grading, and graph routing

3. Dataset Details

Unlike a fixed-corpus classification project, DocuTrust is a document-agnostic platform: any user-uploaded PDF becomes the working dataset at runtime. For development, testing, and the evaluation reported in Section 7, a synthetic but realistic corporate policy handbook was authored and used consistently so that results are fully reproducible.

Field	Details
Dataset Name	Sample Corporate Policy Handbook (project-generated)
Dataset Source	Custom — synthetically generated for this project (scripts/generate_sample_pdf.py)
Documents / Chunks	1 multi-section PDF → 8 chunks after recursive character chunking
Eval Question Set	25 hand-written Q&A pairs (golden_dataset.json): 20 in-corpus, 5 deliberately out-of-corpus
Format	PDF (5 sections: data retention, access control, incident response, remote work, expense policy)
Chunking Strategy	Recursive character split, 800-char chunks with 120-char overlap, page-number tagged
Architecture Note	The ingestion pipeline (parser → chunker → embedder → vector store) is dataset-agnostic and works with any uploaded PDF, not only the bundled sample

4. Project Workflow

The pipeline follows nine sequential stages, from raw PDF upload to a cited, traceable answer:

- **Document Ingestion.** PDF parsed page-by-page (pypdf), split into overlapping chunks, embedded with a bi-encoder, and stored in ChromaDB with content-hash-based document IDs so re-uploading the same file is idempotent.
- **Retrieval (Recall Stage).** On a query, the bi-encoder embeds it and ChromaDB returns the top-20 nearest chunks by cosine distance — a deliberately wide, cheap pass.
- **Reranking & Grading (Precision Stage).** A cross-encoder reranker re-scores the 20 candidates against the query and keeps the top 5. The best score is mapped to a grade: CORRECT, AMBIGUOUS, or INCORRECT, against tuned thresholds.
- **Conditional Routing.** A LangGraph conditional edge routes execution: CORRECT → generate directly; AMBIGUOUS/INCORRECT → rewrite the query and retry retrieval, capped at a maximum number of iterations.
- **Correction Loop.** If retrieval is still weak after the rewrite budget is exhausted, the graph escalates unconditionally to a web-search fallback — guaranteeing the loop can never run forever.

- **Generation with Citations.** The LLM generates an answer strictly from the graded context chunks (or web results), with inline [n] citation markers mapped back to exact source chunk IDs and page numbers.
- **Trace Logging.** Every node transition (retrieve, grade, rewrite, fallback, generate) is appended to a run trace, persisted to MongoDB, and streamed live to the frontend over WebSocket.
- **Evaluation.** A golden dataset of 25 questions is run through both a naive RAG baseline and the full CRAG pipeline; faithfulness and correct-handling-of-out-of-corpus-questions are compared.
- **Deployment.** The full stack (FastAPI backend, MongoDB) is containerised with Docker Compose for one-command setup; a mock-provider mode allows the entire system to run with zero API keys or model downloads.

5. Model / Algorithm Used

5.1 Why a Cross-Encoder for Grading — Not an LLM Self-Judgment

A common alternative design asks the LLM itself, “is this context relevant, yes or no?” This project deliberately uses a dedicated cross-encoder reranker instead, for three concrete reasons:

- **Continuous, thresholdable score.** A cross-encoder outputs a numeric score that can be tuned against fixed thresholds (RELEVANCE_THRESHOLD_CORRECT / _AMBIGUOUS). An LLM's binary yes/no is harder to calibrate and drifts across prompts and model versions.
- **Reproducibility.** The same query/chunk pair produces the same cross-encoder score every run. An LLM's judgment call can vary run to run, which is unacceptable for a grading mechanism the whole control flow depends on.
- **Cost and latency.** A cross-encoder forward pass is far cheaper than an additional LLM round-trip, and this grading step runs on every single query — not just the corrected ones — so it sits on the critical path for the entire system.

5.2 Components Summary

Component	Algorithm / Model
Embedding (recall)	BAAI/bge-small-en-v1.5 — bi-encoder, 384-dim sentence embeddings
Reranking (grading)	BAAI/bge-reranker-v2-m3 — cross-encoder, joint query-document attention
Orchestration	LangGraph StateGraph — explicit finite-state machine with conditional edges
Generation	Claude (Anthropic API) or Llama 3.1 8B (Ollama, local) — pluggable

6. Implementation

6.1 CRAG State Machine — Graph Assembly

```

graph = StateGraph(GraphState)
graph.add_node("retrieve", retrieve_node)
graph.add_node("grade_documents", grade_documents_node)
graph.add_node("generate", generate_node)
graph.add_node("rewrite_query", rewrite_query_node)
graph.add_node("web_fallback", web_fallback_node)
graph.add_node("corrective_generate", corrective_generate_node)

graph.set_entry_point("retrieve")
graph.add_edge("retrieve", "grade_documents")
graph.add_conditional_edges("grade_documents", route_after_grading, {
    "generate": "generate",
    "rewrite_query": "rewrite_query",
    "web_fallback": "web_fallback",
})
graph.add_edge("rewrite_query", "retrieve") # loop back, capped
graph.add_edge("web_fallback", "corrective_generate")

```

6.2 The Grading Decision — Routing Logic with a Termination Guarantee

```

def route_after_grading(state: GraphState) -> str:
    grade = state["grade"]
    iteration_count = state["iteration_count"]

    if grade == "correct":
        return "generate"
    if iteration_count < settings.max_correction_iterations:
        return "rewrite_query"
    # Rewrite budget exhausted -- escalate, never loop again.
    return "web_fallback"

```

This single function answers the most important design question in the project: what stops the correction loop from running forever? Once `iteration_count` reaches the configured cap (default 2), every remaining path is forced to `web_fallback`, regardless of grade — guaranteeing the graph always terminates. This guarantee is pinned down with an executable test, not just a code comment (Section 8).

6.3 Score-to-Grade Mapping

```

def grade_relevance(top_score: float) -> str:
    if top_score >= settings.relevance_threshold_correct: # 0.60
        return "correct"
    if top_score >= settings.relevance_threshold_ambiguous: # 0.30
        return "ambiguous"
    return "incorrect"

```

6.4 Live Trace Streaming (WebSocket)

```
async def run_query_streaming(query: str, on_step):
    state = initial_state(run_id=str(uuid.uuid4()), query=query)
    async for step_output in graph.astream(state):
        for node_name, partial_state in step_output.items():
            for entry in partial_state.get("trace", []):
                await on_step({"type": "trace", "node": node_name, "data": entry})
```

LangGraph's `astream()` yields one event per node transition as it actually happens, which is streamed directly to the frontend over a WebSocket. The interface renders this as a live “Verification Ledger” — the person asking a question watches the agent retrieve, grade, and (when needed) correct itself in real time, rather than waiting for a single opaque final answer.

6.5 Provider Abstraction (Mock / Local / API)

Every model-dependent component — embedder, reranker, LLM, and web search — is written against a shared interface with both a real implementation and a deterministic mock implementation, switched centrally through one settings object:

```
docustrust_provider_mode: str = "mock"      # "mock" | "local"
llm_provider: str         = "mock"         # "mock" | "anthropic" | "ollama"
web_search_provider: str  = "mock"         # "mock" | "tavily"
```

Mock mode requires no API keys, no model downloads, and no live network calls — the entire pipeline, test suite, and evaluation script run fully offline against it. This is the same abstraction boundary a production system would want for swapping embedding models or LLM vendors without touching the graph logic; it is a deliberate design pattern, not a workaround.

7. Results

7.1 Naive RAG vs. CRAG — Evaluation Summary

The system was evaluated on a 25-question golden dataset (20 in-corpus, 5 deliberately out-of-corpus) against the sample policy handbook, comparing the full CRAG pipeline with a naive RAG baseline (retrieve top-k by vector distance, generate directly, no grading or correction).

Metric	Naive RAG	CRAG
Faithfulness (in-corpus, 20 questions)	95.0%	95.0%
Correctly handled (out-of-corpus, 5 questions)	0.0%	100.0%
Avg. latency per query (mock mode)	0.005 s	0.0052 s

7.2 The Key Result: Out-of-Corpus Question Handling

Naive RAG correctly handled **0%** of out-of-corpus questions; CRAG correctly handled **100%**. Naive RAG has no mechanism to recognise that retrieval failed — it presents whatever it retrieves as a confident answer every time, regardless of relevance. CRAG's grading step catches this on every out-of-corpus question in the evaluation, either abstaining explicitly or disclosing a web-search fallback rather than silently answering from irrelevant internal chunks.

Sample Question	In Corpus	Naive RAG	CRAG
How long must transaction records be retained?	Yes	Faithful	Faithful
What is the company's policy on pet adoption leave?	No	Not handled	Handled
What is the boiling point of liquid nitrogen?	No	Not handled	Handled
What is the company's maternity leave policy?	No	Not handled	Handled
Recommend a recipe for chocolate chip cookies	No	Not handled	Handled

7.3 Test Suite Results

25 / 25 automated tests passing (pytest), covering chunking correctness (including a regression test for overlap-duplication bugs), reranker grading-threshold boundaries, and — most importantly — the CRAG graph's routing logic and its iteration-cap termination guarantee, verified as an executable test rather than only a code comment.

7.4 Honest Limitations of the Reported Numbers

All figures above were produced in the project's mock provider mode (deterministic, dependency-free stand-ins for the embedder, reranker, and LLM — see Section 6.5), which is what allows the evaluation to run fully offline and reproducibly. The control-flow result — naive RAG never catches bad retrieval, CRAG always does — is architectural and provider-independent. The absolute faithfulness numbers would shift somewhat with the real embedding and reranker models in local/API mode.

8. Challenges Faced

Several of the challenges below were not implementation bugs in the usual sense, but flaws in how the system's own evaluation measured itself — catching and fixing these is treated as part of the project's deliverable, not a footnote.

- **Flawed Abstention-Scoring Methodology.** An early version of the evaluation checked for correct abstention by matching generic negation words (“not”, “prohibited”) anywhere in the answer text. This produced false positives: a naive RAG answer that extractively quoted an irrelevant chunk happening to contain the word “prohibited” scored as “correctly declining to answer” even though it was confidently presenting unrelated content. Resolved by replacing generic negation matching with explicit abstention-phrase detection (e.g. “don't have enough information”), tied to the actual phrases the system's own providers emit.

- **A Synthetic Confidence Score That Misrepresented Naive RAG.** To give the naive RAG baseline an abstention mechanism comparable to CRAG, an early version derived a fake confidence score from raw vector distance. This was actively misleading: naive RAG has no calibrated relevance signal at all — that absence is its real architectural weakness, and is exactly what motivates CRAG's reranking step. Resolved by removing the synthetic score entirely so the baseline behaves as naive RAG tutorials actually do: always presenting retrieved content, never abstaining.
- **Mock Embedder Baseline Noise.** The dependency-free mock embedder (a hashed bag-of-words and character-trigram scheme, used so the system can run and be tested without HuggingFace model downloads) gives even semantically unrelated text pairs a non-trivial baseline cosine similarity, which initially inflated relevance scores for clearly irrelevant chunks. This was measured empirically rather than assumed, and the relevant grading thresholds and abstention floor were calibrated against the measured baseline, with the limitation documented directly in the architecture notes.
- **Guaranteeing Loop Termination.** An early design risk was an unbounded rewrite-and-retry loop if retrieval kept grading as ambiguous. Resolved with a hard iteration cap enforced in a single routing function, escalating unconditionally to web fallback once the budget is exhausted — and pinned down with a dedicated automated test that asserts the loop can never exceed the configured maximum.
- **Idempotent Document Re-Ingestion.** Re-uploading the same PDF twice initially duplicated its chunks in the vector store, because each chunk's document ID was a fresh random UUID per upload. Resolved by deriving the document ID from a SHA-256 content hash instead, and explicitly clearing any existing chunks for that ID before inserting new ones — making re-uploads of identical content fully idempotent.

9. Conclusion

9.1 What Was Achieved

- Implemented the full Corrective RAG (CRAG) pattern as an explicit LangGraph state machine with three verified execution paths: direct generation, query-rewrite correction, and web-fallback escalation.
- Built and proved a hard termination guarantee on the correction loop, backed by an executable regression test, not just documentation.
- Demonstrated a measurable, honestly-derived advantage over naive RAG: 0% vs. 100% correct handling of out-of-corpus questions on a 25-question golden evaluation set.
- Delivered a full-stack system — FastAPI backend, WebSocket live-trace streaming, ChromaDB vector store, MongoDB trace logging with graceful in-memory fallback, and a custom-designed frontend that visualises the agent's self-correction live.
- Built a provider-abstraction layer (mock / local / API) for every model-dependent component, enabling the entire system, its 25-test suite, and its evaluation to run fully offline and reproducibly.
- Containerised the full stack with Docker Compose for one-command setup.

9.2 Future Improvements

- Add a second grading pass on web-fallback results, which are currently trusted once retrieved rather than re-graded for relevance.
- Add hybrid retrieval (BM25 keyword search combined with vector search) to catch exact keywords and domain jargon the embedding model under-weights.
- Replace the substring-match faithfulness proxy with RAGAS or an LLM-as-judge evaluation pass for subtler hallucination detection.
- Extend evaluation to multi-document corpora; the current evaluation is single-document by design, and relevance grading behaves differently across many overlapping documents.
- Benchmark the real (non-mock) embedding, reranking, and LLM models to report production-representative accuracy and latency figures.

10. References

10.1 Core Papers

- Yan, S.-Q. et al. (2024). Corrective Retrieval Augmented Generation. arXiv:2401.15884.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Nogueira, R., Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085 (cross-encoder reranking foundations).

10.2 Documentation & Tools

- LangGraph Documentation: langchain-ai.github.io/langgraph
- FastAPI Documentation: fastapi.tiangolo.com
- ChromaDB Documentation: docs.trychroma.com
- BGE Reranker (BAAI/bge-reranker-v2-m3): huggingface.co/BAAI/bge-reranker-v2-m3
- BGE Embeddings (BAAI/bge-small-en-v1.5): huggingface.co/BAAI/bge-small-en-v1.5
- Anthropic API Documentation: docs.anthropic.com
- Tavily Search API: tavily.com
- Docker Compose Documentation: docs.docker.com/compose

10.3 Project Repository

- Source Code (GitHub): github.com/DSR001915/docustrust

11. Submission Files

As per the submission format, the following files are included in the final submission package. File naming convention: Devansh_DocuTrust.pdf

#	File Name	Type	Description
1	Devansh_DocuTrust.pdf	PDF	This project report (all 11 sections)
2	DocuTrust_SourceCode.zip	ZIP	Full backend source: ingestion, graph, reranker, LLM, db, tests, eval
3	frontend.zip	ZIP	Frontend: index.html, styles.css, app.js (live verification ledger UI)
4	docker-compose.yml + Dockerfile	Config	One-command containerised setup (backend + MongoDB)
5	requirements.txt / requirements-mock.txt	TXT	Python dependencies, pinned, for full and mock-only setups
6	sample_data/ corporate_policy_handbook.pdf	PDF	Sample document used throughout testing and evaluation
7	docs/EVAL_RESULTS.md + ARCHITECTURE.md	MD	Full evaluation methodology and architecture write-up
8	DocuTrust_Presentation.pptx	PPTX	Project presentation deck (Optional)

Devansh Singh Raghuvanshi | Chandigarh University | B.E. Computer Science & Engineering — Final Year | 01 July 2026

dsraghuvanshi2002@gmail.com | +91 9140405360

Project Link: github.com/DSR001915/docustrust